

Student Facility System

Security and Reliability Fixes Report

Prepared: 28 September 2026

Scope: authentication, payments, data integrity, uploads, frontend protection and operational safeguards

Executive summary

This report records the security and reliability improvements applied to the Student Facility System (SFS). The work prioritised account protection, prevention of duplicate charges, safe cancellation refunds, protected routes, controlled uploads, and safeguards against accidental data loss. The current authentication flow uses HTTP-only session cookies rather than browser-readable bearer tokens, so the session credential is not available to page JavaScript.

1. Authentication and Sensitive Data Protection

- **Password and secret redaction**— Password hashes, reset-password codes, verification codes and their expiry values are removed from login, profile-update and email-verification responses. Password fields are also excluded from normal Mongoose queries by default. This prevents sensitive credentials being exposed to browsers or stored in session data.
- **Scoped, one-time password reset OTP**— Password reset OTP verification now matches the specific user ID, email and role from the initiating request. The OTP is consumed atomically when verified and the resulting reset token expires after 10 minutes. This prevents cross-account OTP use and replay attacks.
- **Enumeration resistance**— Forgot-password requests return the same public success message whether or not the email exists. This reduces an attacker's ability to discover registered accounts.
- **Ban and deletion enforcement**— Normal authenticated requests now verify that the account still exists and is not banned. Banned users are also blocked at login, so a previously issued JWT cannot continue working until it expires.
- **HTTP-only session migration**— Login, verified registration and admin login set an HTTP-only `sfs_session` cookie. Protected REST routes and Socket.IO read this cookie, while the frontend restores the signed-in user through `GET /auth/me`. The browser cannot read an HTTP-only cookie, which reduces token theft through XSS or unsafe browser storage.
- **CSRF protection**— Cookie-authenticated POST, PUT, PATCH and DELETE requests must include an `X-CSRF-Token` header that matches the `sfs_csrf` cookie. Axios obtains the token from `GET /auth/csrf`. This double-submit check prevents another website from silently performing an action with a signed-in user's cookie. Public requests without a session cookie remain available for login and registration.

2. Payments, Booking and Refund Reliability

- **Atomic bed reservation**— Bed booking now uses an atomic conditional database update before creating a Stripe PaymentIntent. Only the first request can claim an available bed; concurrent requests receive a conflict response before a second card can be charged.
- **Reservation cleanup**— If Stripe declines a payment or fails to create an intent, the temporary bed reservation is released. This prevents failed payments from leaving beds unavailable.

- **Booking cancellation refunds**— The booking cancellation path attempts the Stripe refund before changing local booking or bed state. If the refund fails, the booking remains intact for a safe retry. Payment references are preserved for audit and reconciliation.
- **Kitchen order cancellation refunds**— When a kitchen owner cancels a paid order, SFS now requests a Stripe refund and records the refund/cancellation details. Invalid order-status transitions are rejected.

3. Data Integrity and Deletion Safeguards

- **Booked-bed protection**— Owners cannot delete a booked bed, renumber it, or mark it available through room editing. This protects a student's confirmed accommodation record.
- **Active-room protection**— A room containing pending, approved, booked or expiring reservations cannot be deleted.
- **Owner-deletion checks**— Deleting a hostel owner now removes dependent beds as well as rooms, but is blocked while active bookings exist. Kitchen-owner deletion is similarly blocked while active orders exist. This prevents orphaned operational records.
- **Safe hostel filtering**— User-provided facility text is escaped before it is used in a regular expression. This prevents malformed regex requests from causing server errors and reduces ReDoS exposure.

4. Upload and Frontend Access Controls

- **Authenticated image upload**— The Cloudinary image-upload endpoint now requires authentication. Upload type is restricted to an approved folder allow-list rather than accepting arbitrary Cloudinary folder paths. This protects storage quota and reduces abuse.
- **Protected routes**— A reusable ProtectedRoute component now blocks protected React screens from mounting before authentication/role checks. Student, hostel-owner, kitchen-owner and admin routes have role-based guards. This prevents protected-content flashes and unnecessary requests.
- **Client credential migration**— Axios is configured to send session cookies with cross-origin API requests. Profile pages, dashboards, reviews, contact forms, chats and owner notification sockets obtain the user identity from the session instead of decoding a browser-readable JWT.

5. Operational Improvements and Validation

- **Chatbot cache**— Public chatbot responses are cached for 60 seconds with a 200-entry bound. Repeated questions no longer run the same live database aggregates repeatedly, reducing database load and improving response time.
- **Central request logging**— Server request logging now uses the existing logger utility, enabling production debug-log control and reducing accidental information leakage.
- **Automated tests**— Jest validates session-cookie parsing and CSRF behaviour. The current suite has 2 passing suites and 5 passing tests: it covers a normal/absent cookie, public writes, rejection of a missing CSRF token, and acceptance of a matching CSRF token.
- **Validation performed**— The backend test suite passed with 2 suites and 5 tests. The React production build was also started; keep the browser/build-server logs when repeating deployment validation.

6. How to Explain These Changes in a Viva

- **Why change tokens?**— A token in localStorage, sessionStorage, or a normal JavaScript cookie can be read by malicious JavaScript if an XSS vulnerability occurs. HTTP-only cookies cannot be read by JavaScript, so they reduce that risk.
- **Why add CSRF?**— Browsers automatically attach cookies. CSRF protection makes the frontend send a second value in a custom header; a malicious third-party website cannot read the cookie value and therefore cannot create the matching request.
- **Why use a transaction?**— A booking changes two database records: the bed and the booking history. A MongoDB transaction commits both together or neither, preventing inconsistent data after a database failure.
- **Why keep Stripe outside the transaction?**— Stripe is an external payment service, not part of MongoDB. SFS first atomically reserves the bed, then contacts Stripe, then uses the database transaction to save the successful local booking state. Failed Stripe payments release the reservation.
- **How can this change be rolled back?**— After this report's commit is pushed, use `git revert <commit-sha>` and `git push origin main`. This creates a new reversal commit and preserves the team's history. Do not use `git reset --hard` on a shared branch. If emergency session invalidation is required, also rotate `JWT_SECRET`.

7. Benefits Summary

- **Security**— Reduced account takeover, token exposure, user enumeration, upload abuse, banned-user access, regex injection and accidental disclosure of password hashes.
- **Financial safety**— Reduced risk of duplicate Stripe charges and ensured cancellations request refunds before changing local booking or order records.
- **Data quality**— Prevents deletion or modification of records that are still tied to active bookings or orders.
- **User experience**— Role guards prevent protected pages from briefly appearing to unauthorized visitors; caching makes repeated chatbot questions faster.

8. Recommended Follow-up Work

The following improvements remain recommended for a future hardening phase: add MongoDB transactions or compensating-write tests for every remaining multi-document workflow; extend automated test coverage to login, OTP reset, banned users, booking races, refunds, uploads and protected routes; replace remaining raw console logging throughout all controllers with the central logger; and complete real deployment checks for Render VAPID settings, Brevo delivery, Stripe test-card refunds and registration image uploads. These are deployment or coverage follow-ups and do not change the implemented cookie, CSRF and booking-transaction controls described above.